

系统开发工具基础实验报告

第 4 周

姓名：曾奕豪

课程：系统开发工具基础

2026 年 9 月 8 日

仓库：

<https://github.com/hzydy00/Base-of-system-development-tools>

1 课上内容与结果

学习主题

代码质量；元编程；大杂烩；

实验环境

本实验在 Windows 操作系统上完成，命令行环境使用 Git Bash (MINGW64) 与 PowerShell。Python 版本 3.14，代码质量与构建工具链为：ruff 0.16.6（代码格式化与静态检查）、pytest 9.0.3（单元测试）、GNU Make 4.4.1（增量构建）、jq（JSON 处理）、curl（HTTP 请求）、Git 2.53.0.windows.2（版本控制）。

题目一

第 13 题 建立可执行的本地质量门禁

主题：代码质量 建议用时：12—15 分钟

复用第 10 题完成的 greetlab 包，在 q13 中建立一个轻量本地质量检查。

题目要求

- 复制 q10 为 q13；在 pyproject.toml 中加入 ruff 和 pytest 的最小配置。
- 补充一个正常姓名测试和一个空白姓名测试，每个测试包含有效断言。
- 运行 ruff format、ruff check 和 pytest，修复全部问题，不得全局忽略规则。
- 编写 check.sh，依次执行 ruff format --check、ruff check 和 pytest；运行并保证返回 0。

题目一题目要求

```
1 ## TODO: 题目一代码汇总, 分为toml, python, bash三部分
2
3 # pyproject.toml 追加段
4 [tool.ruff]
5 line-length = 88
6 target-version = "py314"
7
8 [tool.pytest.ini_options]
9 testpaths = ["tests"]
10 pythonpath = ["src"]
```

```
1 # tests/test_normal_name.py (新增)
2 import sys
3
4 from greetlab.cli import main
5
6
7 def test_normal_name_prints_greeting(capsys, monkeypatch):
8     monkeypatch.setattr(sys, "argv", ["sdt-greet", "--name", "Alice"
9         ])
10     main()
11     captured = capsys.readouterr()
12     assert captured.out == "Hello, Alice!\n"
```

```
1 # check.sh (新建)
2 #!/usr/bin/env bash
3 set -euo pipefail
4
5 ruff format --check .
6 ruff check .
7 python -m pytest
```

部分实验过程截图

```

$ ruff check .
I001 [*] Import block is un-sorted or un-formatted
--> tests\test_blank_name.py:1:1
1 | / import sys
2 | | import pytest
3 | | from greetlab.cli import main
  | ^
help: Organize imports
1 | import sys
2 | +
3 | import pytest
4 | +
5 | from greetlab.cli import main
  |

Found 1 error.
[*] 1 fixable with the `--fix` option.

ZY@LAPTOP-NOQCE3G0 MINGW64 /d/Course/System_tools_base/week4/q/q13 (main)
$ ruff check --fix .
Found 1 error (1 fixed, 0 remaining).

```

题目一实验过程截图

```

ZY@LAPTOP-NOQCE3G0 MINGW64 /d/Course/System_tools_base/week4/q/q13 (main)
$ ./check.sh
5 files already formatted
All checks passed!

===== test session starts =====
platform win32 -- Python 3.14.0, pytest-9.0.3, pluggy-1.6.0
rootdir: D:\Course\System_tools_base\week4\q\q13
configfile: pyproject.toml
testpaths: tests
plugins: anyio-4.13.0
collected 2 items

tests\test_blank_name.py . [ 50%]
tests\test_normal_name.py . [100%]

===== 2 passed in 0.02s =====

ZY@LAPTOP-NOQCE3G0 MINGW64 /d/Course/System_tools_base/week4/q/q13 (main)
$ echo $?
0

ZY@LAPTOP-NOQCE3G0 MINGW64 /d/Course/System_tools_base/week4/q/q13 (main)
$

```

题目一实验过程截图

实验结果

- ./check.sh 一次通过: ruff format -check (5 files already formatted)
 - ruff check (All checks passed!)
 - pytest (2 passed) → 退出码 0。
- 质量门禁绿灯，本地检查与提交前自检闭环成立。

题目二

第 14 题 让 Make 只重建真正受影响的产物

主题：元编程：构建系统 建议用时：12—15 分钟

在 q14 中按给定内容创建三个源文件，只需完成 Makefile。

给定内容

```
# data.csv
name,value
alpha,2
beta,3
gamma,5

# stats.py
import csv
with open("data.csv") as f:
    total = sum(int(r["value"]) for r in csv.DictReader(f))
open("stats.txt", "w").write(str(total))

# build_report.py
text = open("report.md").read()
total = open("stats.txt").read()
open("report.txt", "w").write(f"{text}\nTotal: {total}\n")

# report.md
# Data Report
```

题目要求

- 创建给定的 data.csv、stats.py、build_report.py 和 report.md。

第 10 页

系统开发工具基础 · 每周课上实验检查题

- 编写 Makefile，包含 all、stats.txt、report.txt 和 clean；准确列出数据文件、脚本和中间产物依赖，并把 all、clean 声明为.PHONY。
- 验证首次 make 生成两个产物；第二次无改动时不执行配方。
- touch data.csv 后再次 make，确认 stats.txt 和 report.txt 重新生成；clean 只删除生成物。

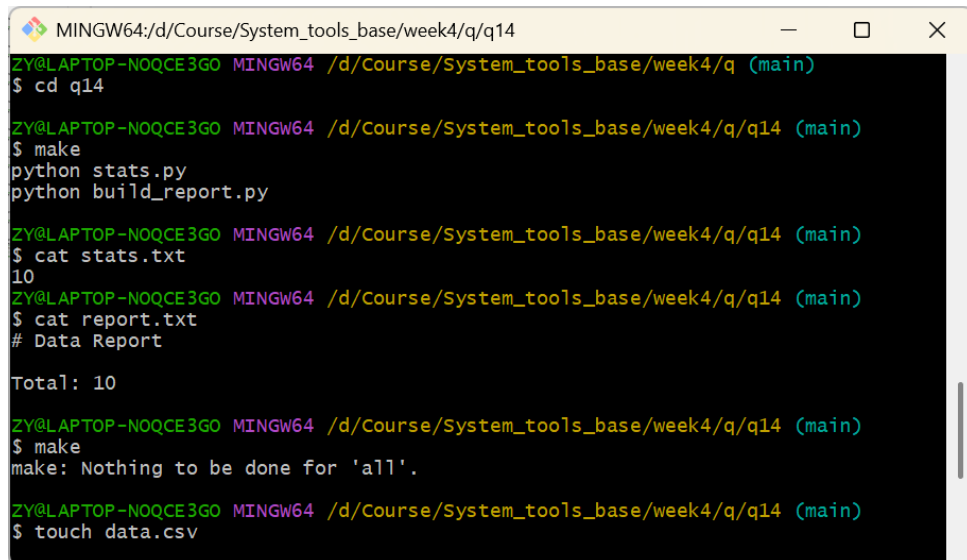
题目二题目要求

```
1 # TODO: 题目二代码汇总，分makefile和bash两部分
2
3 # makefile部分
4 .PHONY: all clean
5
6 all: stats.txt report.txt
7
8 stats.txt: data.csv stats.py
9     python stats.py
```

```
10
11 report.txt: report.md stats.txt build_report.py
12     python build_report.py
13
14 clean:
15     rm -f stats.txt report.txt
```

```
1 # python 部分
2 # 首次构建 —— 预期依次执行两个配方
3 make
4 # 查看产物（预期 stats.txt=10; report.txt 含 "# Data Report" 和 "
   Total: 10"）
5 cat stats.txt
6 cat report.txt
7
8 # 无改动再 make —— 预期 "Nothing to be done for 'all'."
9 make
10
11 # touch 数据文件后再 make —— 预期两个配方重新执行
12 touch data.csv
13 make
14
15 # 清理 —— 预期只删除两个生成物
16 make clean
17 ls
```

部分实验过程截图



```
MINGW64:/d/Course/System_tools_base/week4/q/q14
ZY@LAPTOP-NOQCE3G0 MINGW64 /d/Course/System_tools_base/week4/q (main)
$ cd q14

ZY@LAPTOP-NOQCE3G0 MINGW64 /d/Course/System_tools_base/week4/q/q14 (main)
$ make
python stats.py
python build_report.py

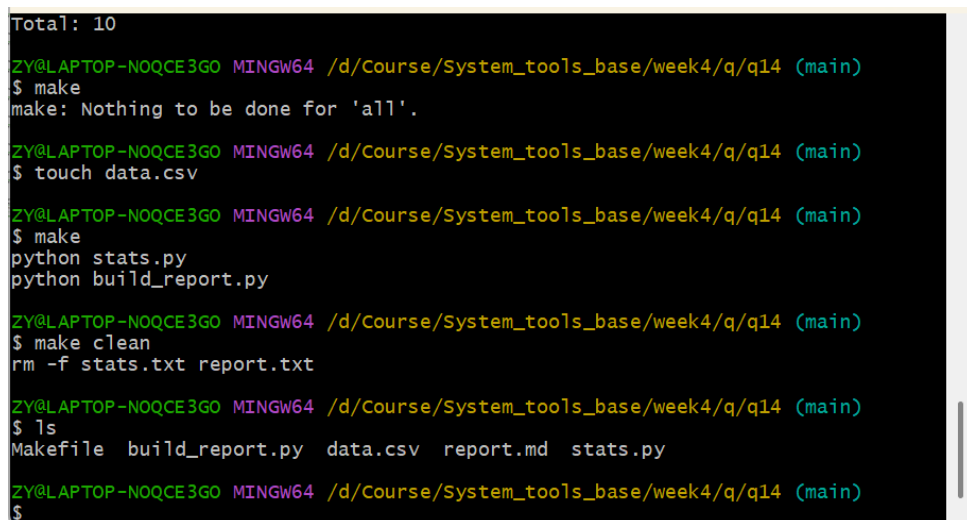
ZY@LAPTOP-NOQCE3G0 MINGW64 /d/Course/System_tools_base/week4/q/q14 (main)
$ cat stats.txt
10
ZY@LAPTOP-NOQCE3G0 MINGW64 /d/Course/System_tools_base/week4/q/q14 (main)
$ cat report.txt
# Data Report

Total: 10

ZY@LAPTOP-NOQCE3G0 MINGW64 /d/Course/System_tools_base/week4/q/q14 (main)
$ make
make: Nothing to be done for 'all'.

ZY@LAPTOP-NOQCE3G0 MINGW64 /d/Course/System_tools_base/week4/q/q14 (main)
$ touch data.csv
```

题目二实验过程截图：首次构建，查看产物



```
Total: 10

ZY@LAPTOP-NOQCE3G0 MINGW64 /d/Course/System_tools_base/week4/q/q14 (main)
$ make
make: Nothing to be done for 'all'.

ZY@LAPTOP-NOQCE3G0 MINGW64 /d/Course/System_tools_base/week4/q/q14 (main)
$ touch data.csv

ZY@LAPTOP-NOQCE3G0 MINGW64 /d/Course/System_tools_base/week4/q/q14 (main)
$ make
python stats.py
python build_report.py

ZY@LAPTOP-NOQCE3G0 MINGW64 /d/Course/System_tools_base/week4/q/q14 (main)
$ make clean
rm -f stats.txt report.txt

ZY@LAPTOP-NOQCE3G0 MINGW64 /d/Course/System_tools_base/week4/q/q14 (main)
$ ls
Makefile build_report.py data.csv report.md stats.py

ZY@LAPTOP-NOQCE3G0 MINGW64 /d/Course/System_tools_base/week4/q/q14 (main)
$
```

题目二实验过程截图：touch 后再构建，清理产物

实验结果

- 首次 make：生成 stats.txt（10）和 report.txt（Total: 10）。
- 无改动再 make：Nothing to be done，配方不执行。
- touch data.csv 后 make：沿依赖链连锁重建两个产物。
- make clean：只删除生成物，源文件保留。

题目三

第 15 题 把本地 API 数据转换为可读报告

主题：大杂烩：API、jq、CLI 约定与 Markdown 建议用时：12—15 分钟

在 q15 中创建给定 JSON 和本地 HTTP 服务，再用 curl 与 jq 生成简短 Markdown 报告。

给定内容

```
[
  {"name": "alpha", "status": "active", "downloads": 120, "version": "1.2.0"},
  {"name": "beta", "status": "inactive", "downloads": 900, "version": "2.0.0"},
  {"name": "gamma", "status": "active", "downloads": 450, "version": "1.5.1"},
  {"name": "delta", "status": "active", "downloads": 450, "version": "0.9.0"},
  {"name": "epsilon", "status": "active", "downloads": 80, "version": "3.1.0"}
]
```

题目要求

- 把给定 JSON 保存为 packages.json，并在 q15 目录运行 python -m http.server 8000。
- 使用 curl -fsS 获取 http://127.0.0.1:8000/packages.json。
- 使用 jq 筛选 status 为 active 且 downloads 不少于 100 的记录，并按 downloads 降序、name 升序排列。
- 编写 api_report.sh，把结果生成 summary.md，包含标题和 name/version/downloads 三列表格。

题目三题目要求

```
1 # TODO: 题目三代码汇总
2 # api_report.sh
3 #!/usr/bin/env bash
4 set -euo pipefail
5
6 {
7     echo "# Package Report"
8     echo ""
9     echo "| name | version | downloads |"
10    echo "|---|---|---|"
11    curl -fsS http://127.0.0.1:8000/packages.json \
12        | jq -r '[] | select(.status == "active" and .downloads >= 100)
13                | sort_by(-.downloads, .name)
14                | .[] | "|" + (.name) + "|" + (.version) + "|" + (.downloads) + "'
15 } > summary.md
16
17 # 终端 A: 启动本地服务（保持运行）
18 python -m http.server 8000
19
20 # 终端 B（另开一个）：取数 + 筛选排序，一步验证
21 curl -fsS http://127.0.0.1:8000/packages.json
```

```

22 curl -fsS http://127.0.0.1:8000/packages.json | jq -r '[] | select
    (.status == "active" and .downloads >= 100)] | sort_by(-.downloads
    , .name) | [] | [.name, .version, .downloads] | @tsv'
23
24 # 运行脚本生成报告
25 ./api_report.sh
26
27 # 查看结果
28 cat summary.md
29
30 # 回到终端 A, Ctrl+C 停止服务

```

部分实验过程截图

The image shows two screenshots of a Windows terminal window. The top screenshot shows the user navigating to the directory `/d/Course/System_tools_base/week4/q/q15` and starting a Python HTTP server on port 8000. The bottom screenshot shows the user running a curl command to fetch data from the local server, followed by running a script `./api_report.sh` and viewing the output in `summary.md`.

```

MINGW64:/d/Course/System_tools_base/week4/q/q15
ZY@LAPTOP-NOQCE3GO MINGW64 /d/Course/System_tools_base/week4/q (main)
$ cd q15

ZY@LAPTOP-NOQCE3GO MINGW64 /d/Course/System_tools_base/week4/q/q15 (main)
$ python -m http.server 8000
Serving HTTP on :: port 8000 (http://[::]:8000/) ...
::ffff:127.0.0.1 - - [08/Sep/2026 10:17:46] "GET /packages.json HTTP/1.1" 200 -
::ffff:127.0.0.1 - - [08/Sep/2026 10:17:56] "GET /packages.json HTTP/1.1" 200 -
::ffff:127.0.0.1 - - [08/Sep/2026 10:18:25] "GET /packages.json HTTP/1.1" 200 -

MINGW64:/d/Course/System_tools_base/week4/q/q15
ZY@LAPTOP-NOQCE3GO MINGW64 /d/Course/System_tools_base/week4/q/q15 (main)
$ curl -fsS http://127.0.0.1:8000/packages.json | jq -r '[] | select(.status =
= "active" and .downloads >= 100)] | sort_by(-.downloads, .name) | [] | [.name,
.version, .downloads] | @tsv'
delta 0.9.0 450
gamma 1.5.1 450
alpha 1.2.0 120

ZY@LAPTOP-NOQCE3GO MINGW64 /d/Course/System_tools_base/week4/q/q15 (main)
$ ./api_report.sh

ZY@LAPTOP-NOQCE3GO MINGW64 /d/Course/System_tools_base/week4/q/q15 (main)
$ cat summary.md
# Package Report

| name | version | downloads |
| --- | --- | --- |
| delta | 0.9.0 | 450 |
| gamma | 1.5.1 | 450 |
| alpha | 1.2.0 | 120 |

ZY@LAPTOP-NOQCE3GO MINGW64 /d/Course/System_tools_base/week4/q/q15 (main)
$

```

题目三实验过程截图

实验结果

- python -m http.server 8000 启动后，curl -fsS 成功取回 packages.json 5 条记录。
- jq 筛选：beta (status=inactive) 与 epsilon (downloads=80 < 100) 被剔除，仅剩 lpha、gamma、delta 三条。

- 排序：downloads 降序为 450、450、120；gamma 与 delta 同为 450 时按 name 升序，delta 排在 gamma 前。
- api_report.sh 生成 summary.md

题目四

第 16 题 修复并交付一个陌生的小型工具仓库

主题：课堂综合测试 建议用时：12—15 分钟

复用第 13 题的项目，在 q16 中完成一次 15 分钟综合检查。

题目要求

- 复制 q13 为 q16 并初始化/继续使用 Git。
- 把问候语实现临时改成始终返回字符串 Hello, name!，运行 check.sh 确认测试失败；随后定位并修复。
- 编写最小 Makefile，包含 check、build 和 clean；check 调用 check.sh，build 生成 wheel。
- 运行 make check 和 make build，计算 wheel 的 SHA-256，并把最终改动提交为一个内容聚焦的 Git 提交。

题目四题目要求

```
1 # TODO: 题目四代码汇总，分为2部分
2
3 # .gitignore
4 __pycache__/_
5 *.pyc
6 .pytest_cache/_
7 .ruff_cache/_
8 *.egg-info/_
9 dist/_
10 build/_
11
12 # Makefile
13 .PHONY: check build clean
14
15 check:
16     ./check.sh
17
18 build:
19     python -m build --wheel
20
21 clean:
```

```

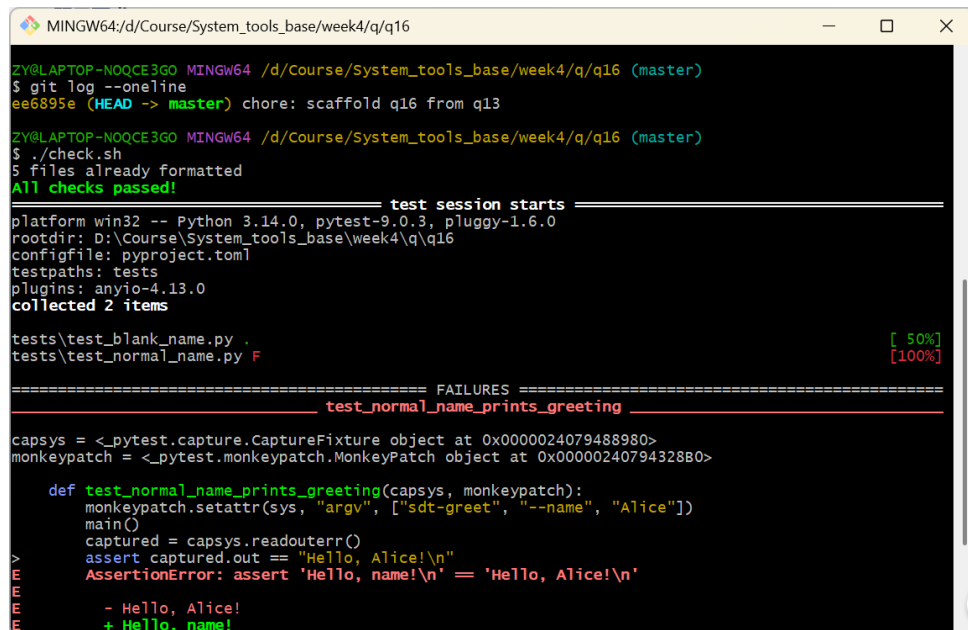
22  rm -rf dist build src/*.egg-info

1  # 红 → 绿
2  ./check.sh                                # 改坏后: 1 failed (红); 修复后: 2
    passed (绿)

3
4  # 构建与校验
5  make check
6  make build
7  sha256sum dist/greetlab_25100021005-0.1.0-py3-none-any.whl
8
9  # 内容聚焦提交 (外部仓库根目录)
10 git add .gitignore
11 git commit -m "chore: ignore python cache and build artifacts"
12 git add week4/q/q16
13 git commit -m "week4 q16: scaffold greetlab from q13"
14 git add week4/q/q16/src/greetlab/cli.py
15 git commit -m "fix: greet with actual name instead of literal string"
16 git log --oneline

```

部分实验过程截图



```

MINGW64/d/Course/System_tools_base/week4/q/q16
ZY@LAPTOP-NOQCE3GO MINGW64 /d/Course/System_tools_base/week4/q/q16 (master)
$ git log --oneline
ee6895e (HEAD -> master) chore: scaffold q16 from q13

ZY@LAPTOP-NOQCE3GO MINGW64 /d/Course/System_tools_base/week4/q/q16 (master)
$ ./check.sh
5 files already formatted
All checks passed!

===== test session starts =====
platform win32 -- Python 3.14.0, pytest-9.0.3, pluggy-1.6.0
rootdir: D:\Course\System_tools_base\week4\q\q16
configfile: pyproject.toml
testpaths: tests
plugins: anyio-4.13.0
collected 2 items

tests\test_blank_name.py . [ 50%]
tests\test_normal_name.py F [100%]

===== FAILURES =====
test_normal_name_prints_greeting

capsys = <pytest.capture.CaptureFixture object at 0x0000024079488980>
monkeypatch = <pytest.monkeypatch.MonkeyPatch object at 0x00000240794328B0>

def test_normal_name_prints_greeting(capsys, monkeypatch):
    monkeypatch.setattr(sys, "argv", ["sdt-greet", "--name", "Alice"])
    main()
    captured = capsys.readouterr()
    assert captured.out == "Hello, Alice!\n"
> assert 'Hello, name!\n' == 'Hello, Alice!\n'
E       AssertionError: assert 'Hello, name!\n' == 'Hello, Alice!\n'
E
E       - Hello, Alice!
E       + Hello, name!

```

题目四实验过程截图: 运行 check.sh, ruff 照常通过, 但 pytest 失败

```

ZY@LAPTOP-NOQCE3GO MINGW64 /d/Course/System_tools_base/week4/q/q16 (main)
$ make check
./check.sh
5 files already formatted
All checks passed!

===== test session starts =====
platform win32 -- Python 3.14.0, pytest-9.0.3, pluggy-1.6.0
rootdir: D:\Course\System_tools_base\week4\q\q16
configfile: pyproject.toml
testpaths: tests
plugins: anyio-4.13.0
collected 2 items

tests\test_blank_name.py . [ 50%]
tests\test_normal_name.py . [100%]

===== 2 passed in 0.02s =====

ZY@LAPTOP-NOQCE3GO MINGW64 /d/Course/System_tools_base/week4/q/q16 (main)
$ make build
python -m build --wheel
* Creating isolated environment: venv+pip...
* Installing packages in isolated environment:
  - setuptools>=68
* Getting build dependencies for wheel...
running egg_info
creating src\greetlab_25100021005.egg-info
writing src\greetlab_25100021005.egg-info\PKG-INFO
writing dependency_links to src\greetlab_25100021005.egg-info\dependency_links.txt
writing entry points to src\greetlab_25100021005.egg-info\entry_points.txt
writing top-level names to src\greetlab_25100021005.egg-info\top_level.txt
writing manifest file 'src\greetlab_25100021005.egg-info\SOURCES.txt'
reading manifest file 'src\greetlab_25100021005.egg-info\SOURCES.txt'
writing manifest file 'src\greetlab_25100021005.egg-info\SOURCES.txt'

```

题目四实验过程截图: 修复后通过 makefile 运行 check 和 build, 正常

```

MINGW64/d/Course/System_tools_base

ZY@LAPTOP-NOQCE3GO MINGW64 /d/Course/System_tools_base/week4/q/q16 (main)
$ sha256sum dist/greetlab_25100021005-0.1.0-py3-none-any.whl
7cc3ccd4e6a247686780da307bc6c90692712c32f6e98c02ec5f09f02227f25f *dist/greetlab_25100021005-0.1.0-py3-none-any.whl

ZY@LAPTOP-NOQCE3GO MINGW64 /d/Course/System_tools_base/week4/q/q16 (main)
$ cd "D:\Course\System_tools_base"

ZY@LAPTOP-NOQCE3GO MINGW64 /d/Course/System_tools_base (main)
$ git log --oneline
dcb8219 (HEAD -> main) fix: greet with actual name instead of literal string
39d3ced week4 q16: scaffold greetlab from q13
afd047f chore: ignore python cache and build artifacts

```

题目四实验过程截图: 查看 wheel 的 SHA-256 值, 并提交最终改动

实验结果

- 改坏后, ./check.sh 红; 修复后绿 (2 passed)
- 编写 makefile, 通过 make build 产出 wheel, 并显示 SHA-256 值
- 外部仓库三步提交 (chore → scaffold → fix, 仅 1 行改动)

课上阶段提交记录

[图片待补充] week\Week__commit_kcheng.png

课上阶段 commit 记录

2 课后练习与结果

实例 1

实验题目: 此处填写实例一的实验题目与要求描述。

1 # TODO: 实例1代码汇总

运行结果:

[图片待补充] *week\Week_ex1.png*
实例一运行结果

实例 2

实验题目：此处填写实例二的实验题目与要求描述。

1 # TODO：实例2代码汇总

运行结果：

[图片待补充] *week\Week_ex2.png*
实例二运行结果

实例 3

实验题目：此处填写实例三的实验题目与要求描述。

1 # TODO：实例3代码汇总

运行结果：

[图片待补充] *week\Week_ex3.png*
实例三运行结果

实例 4

实验题目：此处填写实例四的实验题目与要求描述。

1 # TODO：实例4代码汇总

运行结果：

[图片待补充] *week\Week_ex4.png*
实例四运行结果

实例 5

实验题目：此处填写实例五的实验题目与要求描述。

1 # TODO：实例5代码汇总

运行结果：

[图片待补充] *week\Week_ex5.png*
实例五运行结果

实例 6

实验题目：此处填写实例六的实验题目与要求描述。

```
1 # TODO: 实例6代码汇总
```

运行结果：

[图片待补充] *week\Week_ex6.png*
实例六运行结果

实例 7

实验题目：此处填写实例七的实验题目与要求描述。

```
1 # TODO: 实例7代码汇总
```

运行结果：

[图片待补充] *week\Week_ex7.png*
实例七运行结果

实例 8

实验题目：此处填写实例八的实验题目与要求描述。

```
1 # TODO: 实例8代码汇总
```

运行结果：

[图片待补充] *week\Week_ex8.png*
实例八运行结果

实例 9

实验题目：此处填写实例九的实验题目与要求描述。

```
1 # TODO: 实例9代码汇总
```

运行结果：

[图片待补充] *week\Week_ex9.png*
实例九运行结果

实例 10

实验题目：此处填写实例十的实验题目与要求描述。

1 # TODO：实例 10 代码汇总

运行结果：

[图片待补充] *week\Week_ex10.png*
实例十运行结果

实例 11

实验题目：此处填写实例十一的实验题目与要求描述。

1 # TODO：实例 11 代码汇总

运行结果：

[图片待补充] *week\Week_ex11.png*
实例十一运行结果

课后阶段提交记录

[图片待补充] *week\Week_commit_kehou.png*
课后阶段 *commit* 记录

3 解题感悟

此处填写解题感悟。